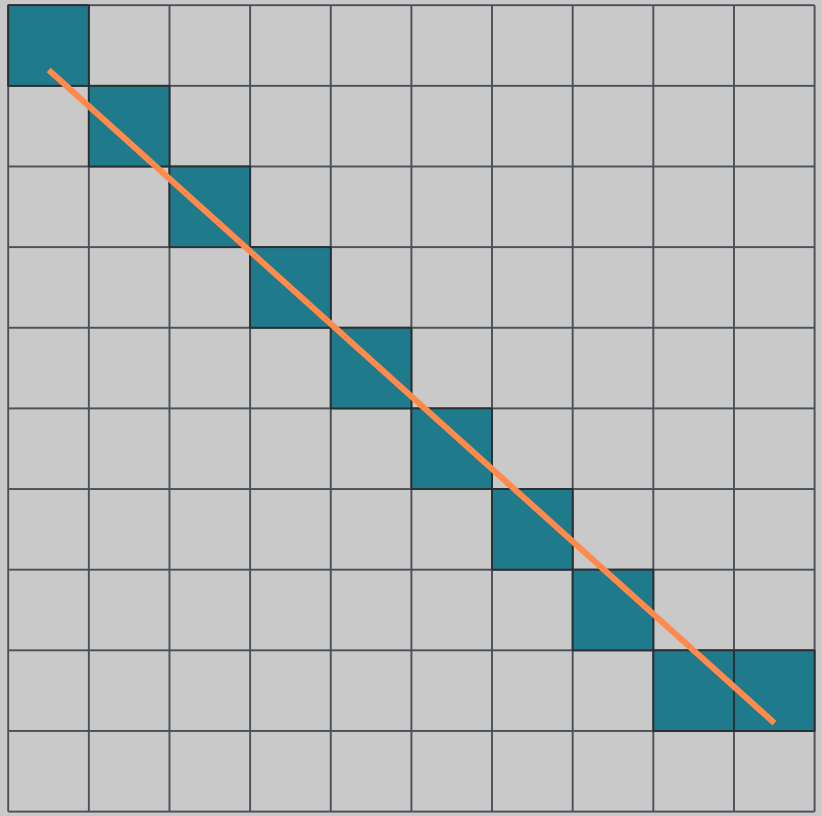
Faculty of Computers and Information

Computer Graphics
IT311

Chapter 1 — Introduction

Rasterization · Transformations · 3D Viewing · Simulation · Virtual Reality



Compiled by: Dr. Marwa Hassan Salama

Course Aim: What You Will Achieve This Semester

By the end of the semester you will understand how a computer turns the description of a scene into an array of colored pixels — and you will be able to implement each step yourself in Python, starting from the pixel.

1 Understand the pipeline

Follow a scene from modeling and transformation, through clipping and viewport mapping, to rasterization and display.

2 Rasterize shapes

Derive and implement the algorithms that draw lines, circles, ellipses and curves, and that fill areas.

3 Clip to a window

Restrict points, lines and polygons to a rectangular viewing window before they are drawn.

4 Transform in 2D and 3D

Use matrices and vectors to translate, scale, rotate and combine transformations.

5 View a 3D scene

Represent 3D objects, place a virtual camera, and project the scene onto a 2D image.

6 Animate and immerse

Explore particle systems and physical simulation, and the low-latency pipeline of virtual reality.

Source: IT311 Computer Graphics book — About This Book, Learning Objectives

Course Content: The Whole Semester

Chapters 1 – 6: from the pixel

Chapters 7 – 9: transformations and viewing

Chapters 10 – 11: simulation and VR

1 Chapter 1

Introduction

2 Chapter 2

Line Drawing Algorithms

3 Chapter 3

Circle and Ellipse Drawing Algorithms

4 Chapter 4

Drawing Parametric Curves and Splines

5 Chapter 5

Filling Algorithms

6 Chapter 6

Clipping Algorithms

7 Chapter 7

2D Transformations

8 Chapter 8

3D Transformation & Vector Space

9 Chapter 9

3D Representation and Projections

10 Chapter 10

Particle Systems and Physical Simulation

11 Chapter 11

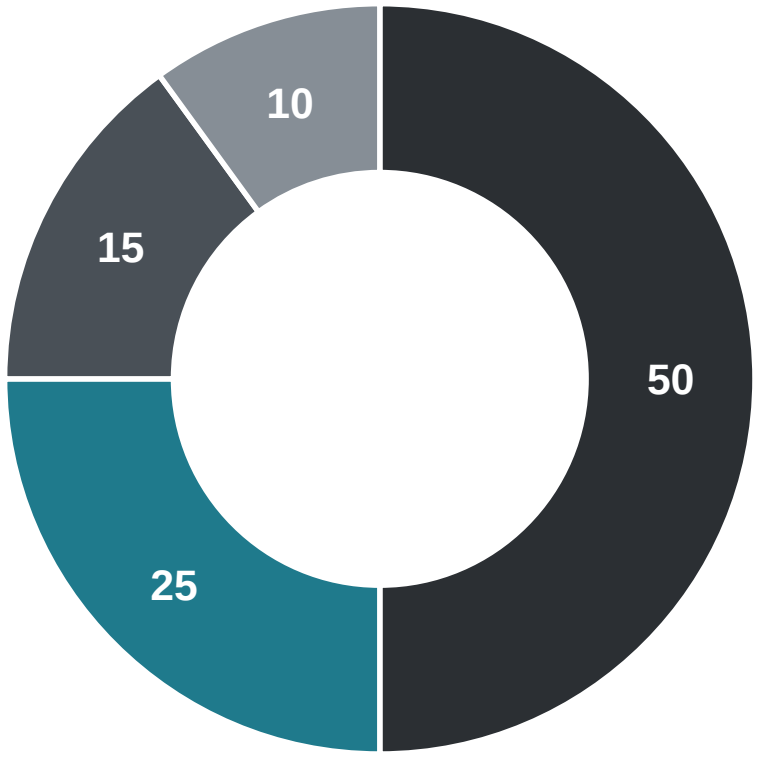
Virtual Reality

A Appendix

Python listings, function index, formulas

Source: IT311 Computer Graphics book — Table of Contents

Grade Distribution



50	**Final Exam** 50% of the total grade
25	**Midterm** 25% of the total grade
15	**Practical** 15% of the total grade
10	**Activities** 10% of the total grade

Total: 100 marks

Learning Objectives

After studying this chapter you should be able to:

1

Describe the main application areas of computer graphics and the hardware and software of a graphics system.

2

Explain the stages of the graphics pipeline, from a scene description to the pixels on the screen.

3

Distinguish a raster image from a vector description, and pixel coordinates from world coordinates.

4

Describe the RGB, RGBA and HSV color models and the meaning of the alpha value.

5

Explain how basic shapes are drawn as strokes and fills, and how transformations, windows, viewports and clipping decide what appears on the screen

.

6

Explain hierarchical modeling with a scene graph and outline how these ideas extend from 2D to 3D.

What Is Computer Graphics?

Definition

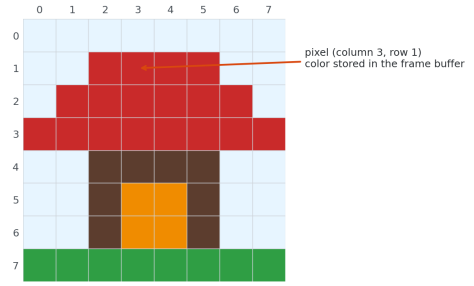
Computer graphics turns a description of a scene into an array of colored pixels.

Input: a description of a scene

- Basic shapes (strokes and fills)
- Coordinates in world space
- Attributes such as color

Computer graphics

transformations → window → clipping → viewport → rasterization



Output: an array of colored pixels

Why it matters. Computer graphics is rarely an end in itself; it is the technology that makes countless other fields visual — engineering, entertainment, medicine, user interfaces, scientific visualization, and virtual reality.

Source: IT311 Computer Graphics book, Chapter 1 — Chapter Summary

Introduction: Stepping Back

Before working through the drawing, filling, clipping, and transformation algorithms in the chapters that follow, it helps to step back and see how all the pieces fit together.

This chapter lays out the basic vocabulary and ideas that the rest of the book builds on.

None of the algorithms are covered here in full detail — that is the job of the chapters that follow — but by the end of this chapter, the motivation and the terminology behind each of them should be clear.

The chapter covers:

1. What computer graphics is used for
2. What hardware and software make it possible
3. What a digital image actually is
4. How coordinates and colors are represented
5. What counts as a basic shape
6. How shapes are moved and combined into larger scenes
7. How all of this extends into three dimensions

1 Applications of Computer Graphics

Computer graphics is rarely an end in itself; it is the technology that makes countless other fields visual.

Engineering & Architecture

Computer-aided design (CAD) systems let a designer build and inspect a product or a building on screen long before anything physical exists.

Entertainment

The same mathematics that draws a line or fills a polygon scales up to the photorealistic scenes of an animated film, the real-time worlds of a video game, and visual effects that composite computer-generated objects into live-action footage.

Medicine

Graphics turns raw scanner data — from an MRI, a CT scan, or an ultrasound — into images a physician can read, often as interactive three-dimensional reconstructions rather than flat slices.

Graphical User Interfaces

Every GUI, from a phone's home screen to a desktop operating system, is itself a real-time graphics application, redrawing windows, icons, and text many times a second.

Scientific & Business Visualization

The same tools turn large, otherwise unreadable datasets into charts, plots, and interactive visualizations that make patterns visible at a glance.

Virtual & Augmented Reality

VR and AR render a scene twice per frame, once for each eye, and update it fast enough to keep pace with a user's head movements.

Every one of these applications is built from the same small set of ideas: coordinates, colors, shapes, transformations, and the pipeline that turns all of them into pixels.

Source: IT311 Computer Graphics book, Chapter 1

2 Graphics Hardware: The GPU

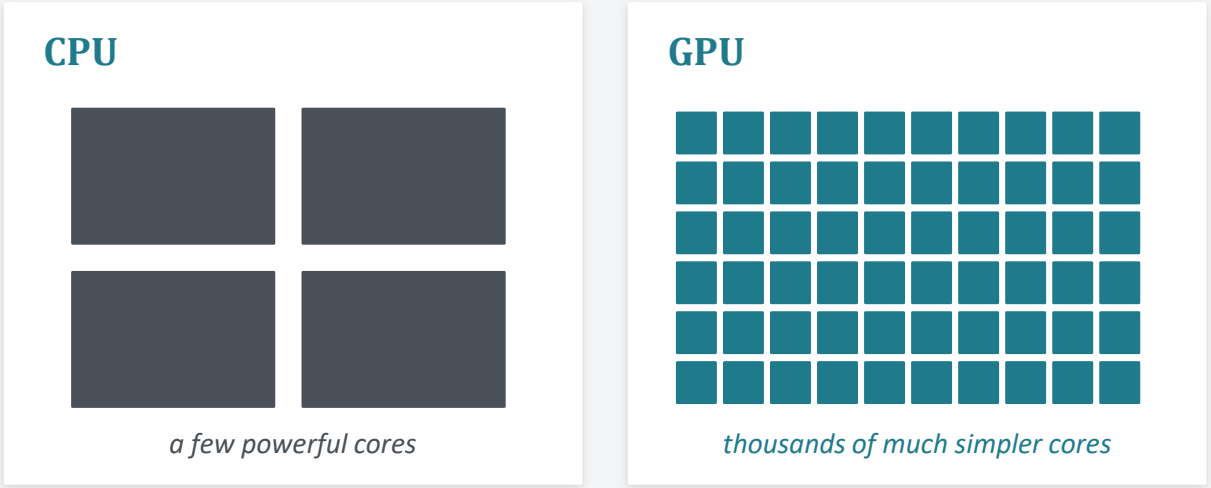




Illustration added for clarity (not in the book).

Definition. The graphics processing unit, or GPU, is a processor built specifically for graphics work.

CPU vs. GPU. Unlike a general-purpose CPU, which executes a single stream of instructions very quickly, a GPU contains thousands of much simpler cores that all run the same operation on different pieces of data at once — exactly the kind of work needed to compute a color for every one of the millions of pixels on a screen, many times a second.

VRAM. The values a GPU computes are written into a dedicated block of video memory, or VRAM, that holds the frame buffer described later in this chapter; keeping this memory physically close to the GPU is what makes real-time rendering possible.

2 Input and Output Devices

A graphics system also needs a way for a user to supply input and a way to present its output.

Input devices

1. **Keyboard and mouse**
2. **Graphics tablet or stylus**
   for freehand drawing
3. **Touch screen**
   combines input and display in one surface
4. **Scanners or digital cameras**
   bring existing images into the system

Output devices

1. **Display (LCD, LED, or OLED monitor)**
   remains the most common device
2. **Printer or plotter**
   for a physical copy
3. **Projector**
   for a shared view
4. **Head-mounted display (HMD)**
   for virtual and augmented reality; the system must render two closely related images at once, one per eye

3 Graphics Software

Graphics software is generally organized in two layers.

High-level applications
Adobe Photoshop, AutoCAD, Blender
Present familiar tools — brushes, layers, modeling operations — without requiring any knowledge of what happens underneath.

Low-level graphics libraries
OpenGL, Direct3D, Vulkan
Sit close to the hardware and expose the GPU's capabilities directly, in terms of vertices, textures, and shader programs; the layer that game engines and other performance-critical applications are built on.

GPU (graphics hardware)
Every pixel the high-level layer produces is put on screen through calls that ultimately reach a low-level graphics library.

This book works one level below

Rather than calling a ready-made drawing library, it derives the pixel-level algorithms that a library like that would use internally.

The code is written in Python around a single, generic `set_pixel(x, y, color)` function, so the same algorithm can be connected to whatever drawing surface is convenient — a Pillow image, a Tkinter canvas, or a raw pixel array — without changing a single line of the algorithm itself.

4 The Graphics Pipeline: An Overview

Every image produced by a graphics system, no matter how complex, is the result of the same basic sequence of steps, usually called the graphics pipeline.

1 Modeling	2 Geometric transformations	3 Window → viewport	4 Clipping	5 Rasterization
Defining a shape's geometry numerically, in whatever coordinate system is most convenient for describing it.	Translation, rotation, scaling, and the like — to place, orient, and size the shape correctly within the scene.	The scene is expressed relative to the view window (the region to be displayed) and mapped into a viewport, the corresponding region of the screen or image.	Removes or trims whatever falls outside the view window, so that only the visible portion of each shape is ever processed further.	Converted from an exact geometric description into a specific set of colored pixels, the only thing a screen can actually display.

model → transform → map to a viewport → clip → rasterize → display

4 The Pipeline Across This Book

This same sequence — model, transform, map to a viewport, clip, rasterize — reappears, largely unchanged, throughout the rest of this book:

Rasterization Chapters 2 – 3	The line- and circle-drawing algorithms are rasterization in its purest form.
Clipping Chapter 6	The clipping chapter works out the middle step in full detail.
Transformations Chapter 7	The transformation chapter formalizes the matrices that carry out the second step.

Seeing the whole pipeline laid out once, here, makes it much easier to see where each later chapter's algorithms fit.

Source: IT311 Computer Graphics book, Chapter 1

5 Raster Images and the Frame Buffer

Pixel

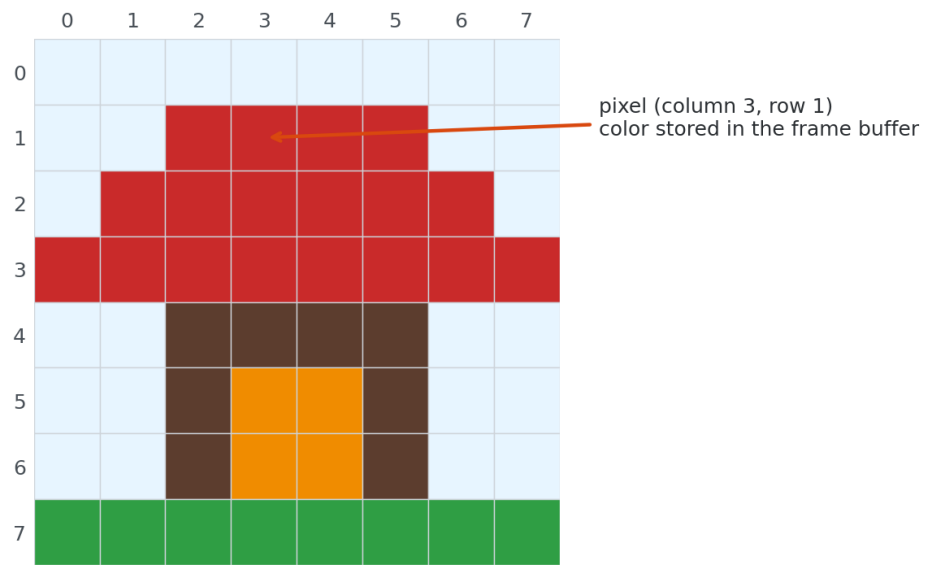
A computer screen is a grid of tiny picture elements, or pixels, each of which can be set to a single color at any moment.

Frame buffer

The colors of every pixel are kept in a block of memory called a frame buffer, and redrawing the screen simply means updating the values stored there.

Raster image

An image built this way, pixel by pixel, is called a raster image.



Historical note. Before raster screens became fast and cheap, some displays literally swept an electron beam along the lines that were to be drawn, redrawing a stored list of lines over and over so quickly that the eye perceived a stable picture. That older approach highlights the second, very different way of describing an image: not as colored pixels, but as a list of geometric objects.

5 Painting Programs vs. Drawing Programs

Painting program (raster)

A program that lets a user paint directly onto a pixel grid. Once a pixel has been colored, the program has no memory of what shape put it there, so erasing a shape only reveals whatever was underneath, pixel by pixel.

Drawing program (vector description)

Represents the picture as a list of shapes — lines, circles, curves, polygons — each with attributes such as color, thickness, or fill pattern. Because each shape remains a distinct object, it can be moved, resized, restyled, or deleted independently of everything else in the picture.

Why vector descriptions are useful

Compact

A shape needs only its defining coordinates, not a color value for every pixel it covers.

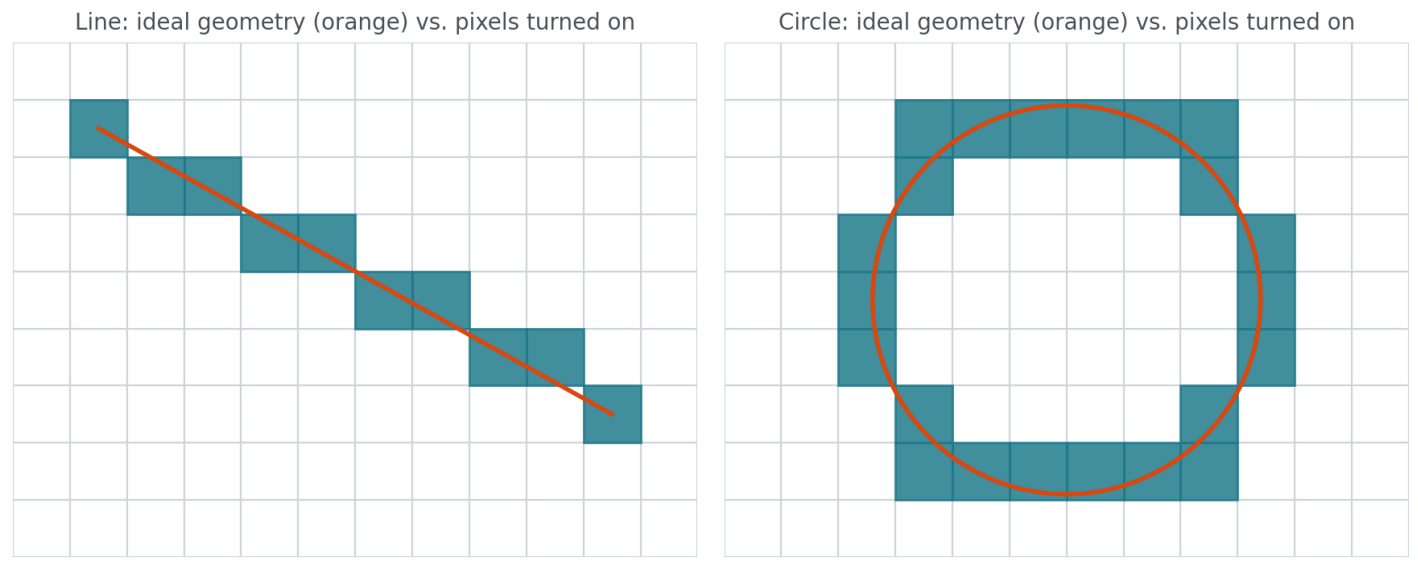
Scalable

They scale to any size without loss of quality — which is why formats such as SVG are so useful for icons and illustrations that need to look sharp at any resolution.

Source: IT311 Computer Graphics book, Chapter 1

5 Rasterization

But a screen can only ever display pixels. Somewhere between the two representations, every line, circle, curve, and filled region has to be converted into a specific set of colored pixels.



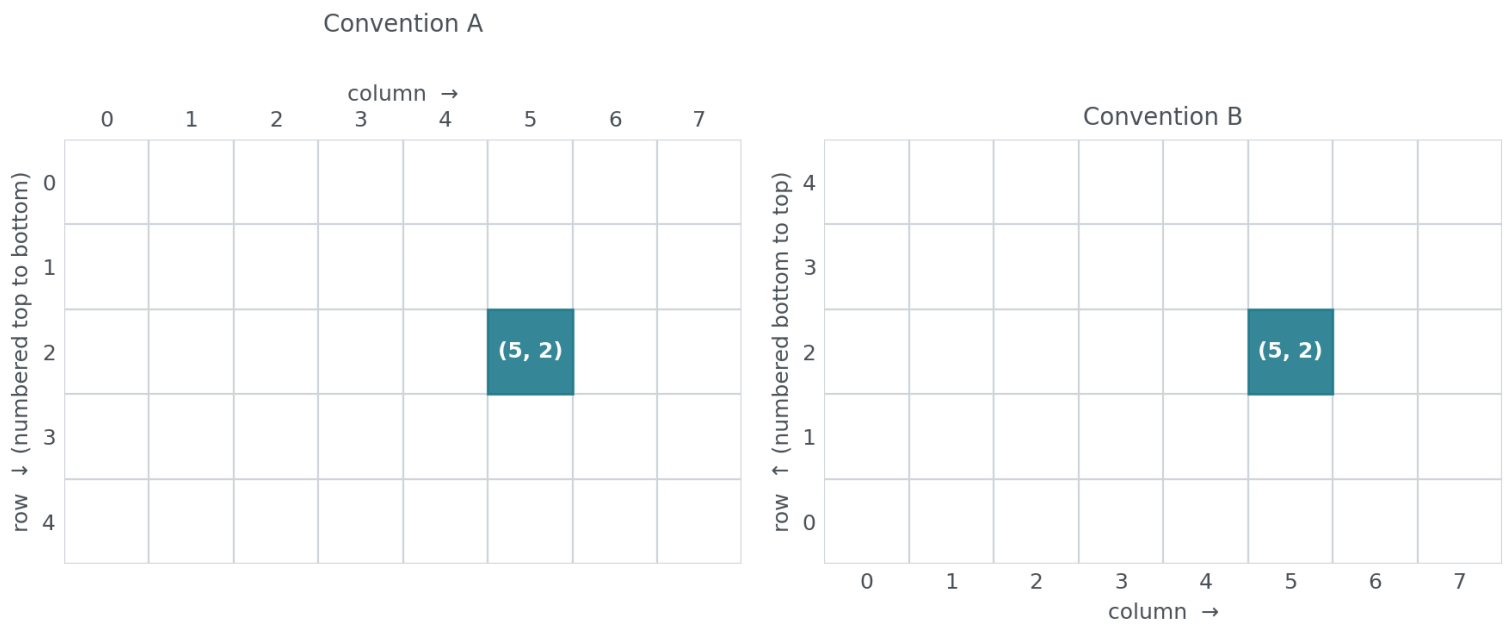
Rasterization

That conversion process is called rasterization: given a clean geometric description of a shape, decide precisely which pixels must be turned on, and to what color, to best approximate that shape on a grid that has no true diagonal or curved lines at all.

It is exactly what the line-, circle-, curve-drawing, and filling algorithms in the next few chapters are designed to do.

6 Pixel Coordinates

To talk about points precisely we need a coordinate system, which is simply a consistent way of assigning a pair of numbers to every point.

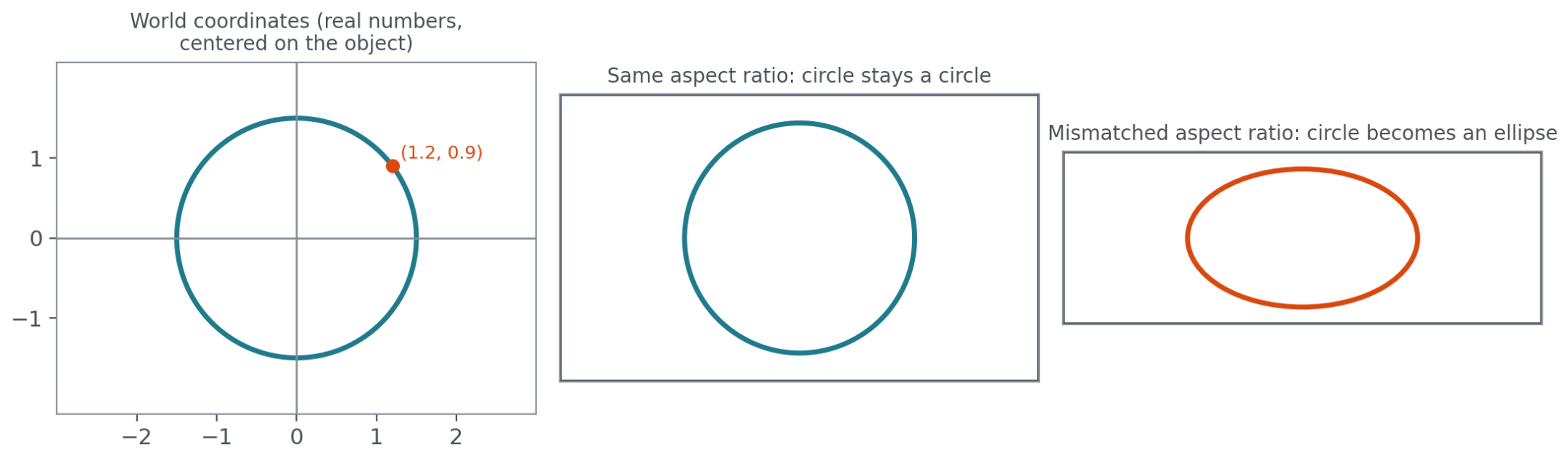


The most direct coordinate system for a raster image numbers pixels by column and row: a pixel is identified by an integer pair giving which column and which row contains it.

Columns are conventionally numbered left to right starting at zero.

Rows are less standard — some systems number them top to bottom, others bottom to top — a common source of bugs when porting drawing code between graphics libraries.

6 World Coordinates and Aspect Ratio



World coordinates

For anything beyond addressing individual pixels, it is far more convenient to work with an abstract, real-number coordinate system chosen to fit the problem at hand — and to map that system onto the pixel grid only at the very last step.

Aspect ratio

Keeping the width-to-height ratio of that mapping, its aspect ratio, consistent between the chosen coordinate system and the physical drawing area avoids stretching or squashing the image; a circle defined mathematically must still look like a circle once it reaches the screen, not an ellipse.

Source: IT311 Computer Graphics book, Chapter 1

7 Color Models: RGB

Color needs its own numerical representation. The most common scheme, RGB, specifies a color by the intensities of red, green, and blue light that combine to produce it.

3
channels: R, G, B

8 bits
per channel (24-bit color)

0 – 255
integer value of each component

256
possible intensities per channel

In the everyday 24-bit representation, each of the three components is an 8-bit integer from 0 to 255, giving 256 possible intensities per channel — roughly as many gradations as the human eye can distinguish, which is why this format is the de facto standard for screens.



Red
(255, 0, 0)

Green
(0, 255, 0)

Blue
(0, 0, 255)

Yellow
(255, 255, 0)

White
(255, 255, 255)

Black
(0, 0, 0)

7 Color Models: HSV (HSB) and HSL

Other models describe the same set of colors in more intuitive terms: HSV (or HSB) and HSL separate color into hue, saturation, and either value or lightness.

Hue: a circle from red back to red



Hue

The basic spectral color, arranged around a circle from red through yellow, green, cyan, blue, and magenta back to red.

Saturation

How far the color is from a shade of gray.

Value or lightness

How bright or dark the color is.

7 Transparency: The Alpha Value (RGBA)

A fourth value, alpha, is frequently added to represent transparency: an alpha of one means fully opaque, zero means fully invisible, and anything in between blends the foreground color with whatever color already lies behind it.

This blending is computed independently for each color channel as a weighted average of the foreground and background values, weighted by alpha:

$$C_{\text{result}} = \alpha \cdot C_{\text{foreground}} + (1 - \alpha) \cdot C_{\text{background}}$$

(for each of R, G and B separately)

Illustration: red foreground over a blue background



α = 1
(255, 0, 0)

α = 0.75
(191, 0, 64)

α = 0.5
(128, 0, 128)

α = 0.25
(64, 0, 191)

α = 0
(0, 0, 255)

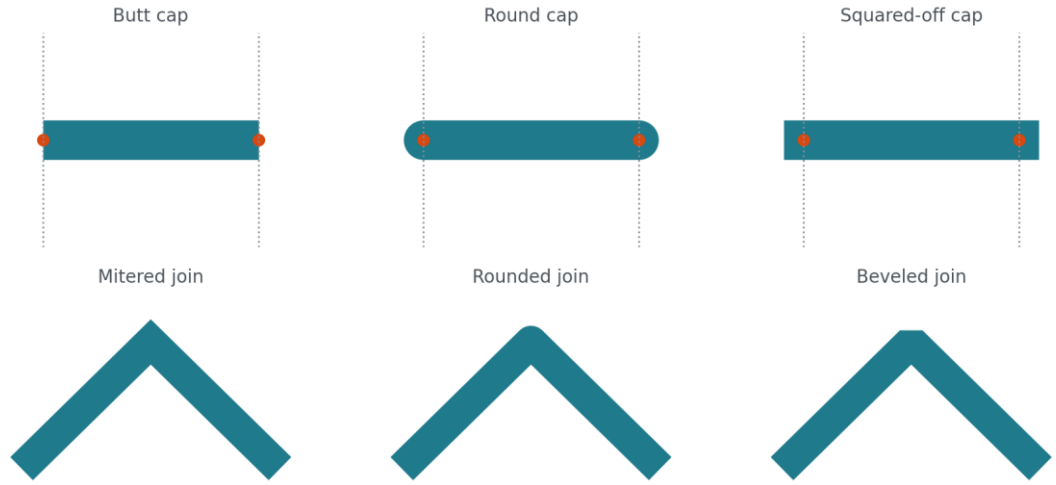
It is how overlapping shapes, soft edges, and layered graphics of every kind are actually rendered.

8 Basic Shapes: Line Segments

Almost every 2D graphics system is built around a small set of basic shapes: line segments, rectangles, circles and ovals, and more general polygons and curves.

Even a single line segment carries decisions

- How wide it should be drawn.
- What shape its two end caps should take — a flat butt cap, a rounded cap, or a squared-off cap that extends slightly past the endpoint.
- Where lines meet at a corner, whether that join should be mitered to a sharp point, rounded, or beveled flat.



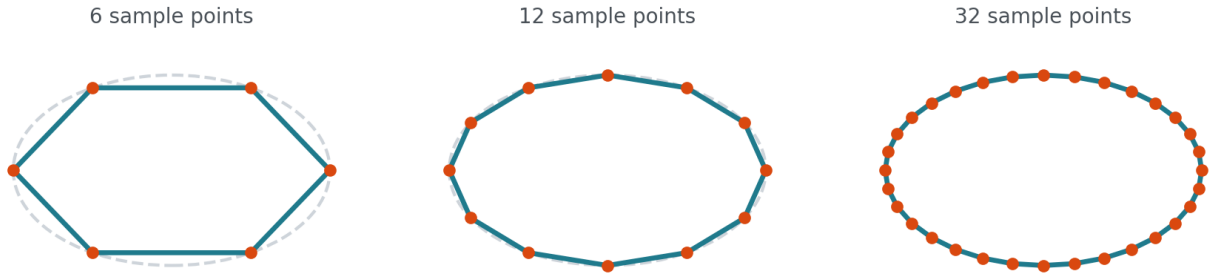
8 Basic Shapes: Rectangles and Ovals

Rectangle

A rectangle can be given either by two opposite corners or by one corner together with a width and height, and its corners can be rounded by replacing them with small elliptical arcs.

Oval

An oval is defined by a center point and a horizontal and vertical radius, with a circle simply being the special case where the two radii are equal.



If a graphics system has no built-in oval command, the same shape can be approximated closely enough by sampling points around it at regular angles and connecting them with straight line segments.

8 Stroke and Fill



Two separate operations

Stroking its outline — where the questions about line width, caps, and joins come in.

Filling its interior with a solid color, a gradient, or a pattern.

More complex figures are built as polygons, defined by a sequence of vertices connected edge to edge, or as paths, which may mix straight segments with curved ones.

Underneath every ready-made library command sits a pixel-level algorithm that has to decide exactly which pixels an outline passes through and exactly which pixels lie inside a closed region. Working out those algorithms directly — for lines, circles, splines, and filled polygons — is the subject of the four chapters that open this book.

9 Geometric Transformations: the Affine Form

A geometric transform takes the coordinates of a point and produces new coordinates for it. The transforms used throughout 2D graphics can all be written in the general affine form:

$$x' = a \cdot x + b \cdot y + e$$
$$y' = c \cdot x + d \cdot y + f$$

for six constants a through f

Same transform in matrix (homogeneous) form

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & e \\ c & d & f \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Translation

Shifts every point by a fixed horizontal and vertical amount, e and f, leaving everything else unchanged.

$$x' = x + e, \quad y' = y + f$$

Rotation

Turns every point through a fixed angle θ about the origin, following the familiar sine-and-cosine formulas.

$$x' = x\cos\theta - y\sin\theta$$
$$y' = x\sin\theta + y\cos\theta$$

Scaling

Multiplies the x- and y-coordinates by separate factors, stretching or shrinking a shape (a negative factor also reflects it).

$$x' = sx \cdot x, \quad y' = sy \cdot y$$

Shear

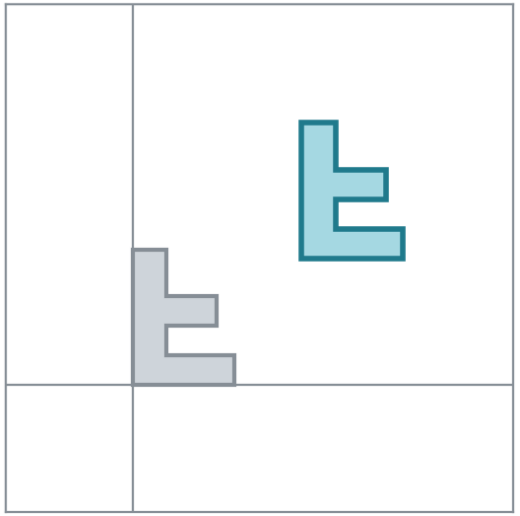
Tilts a shape by displacing each point horizontally (or vertically) by an amount proportional to its other coordinate.

$$x' = x + k \cdot y, \quad y' = y$$

Source: IT311 Computer Graphics book, Chapter 1

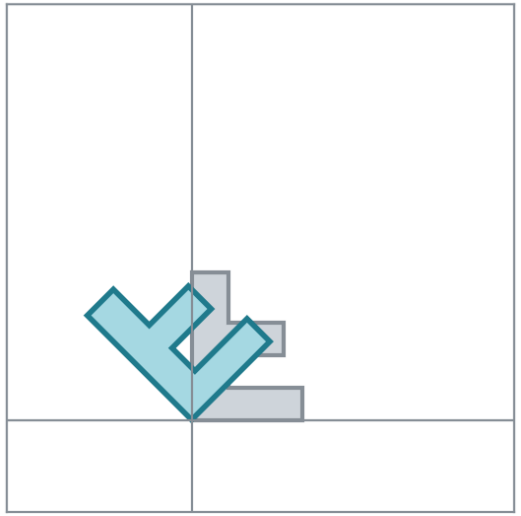
9 The Named Transforms in Action

Translation



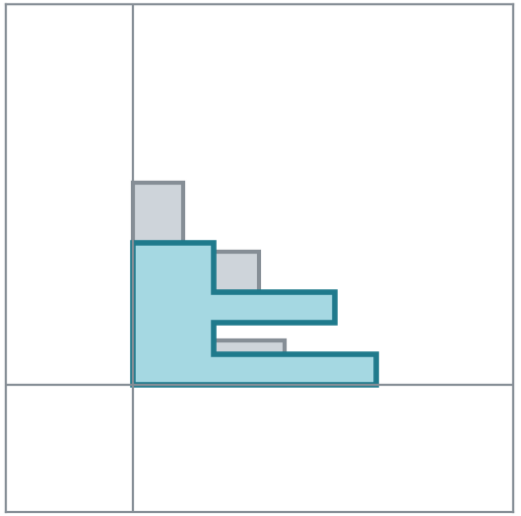
$x' = x + 2,\ y' = y + 1.5$

Rotation (45° about origin)



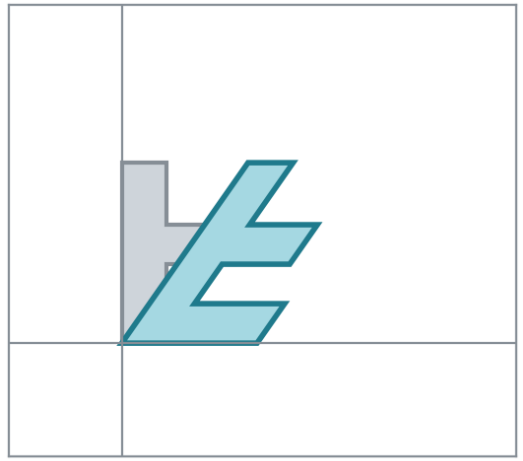
$x' = x\cos\theta - y\sin\theta,\ y' = x\sin\theta + y\cos\theta$

Scaling (sx = 1.6, sy = 0.7)



$x' = sx \cdot x,\ y' = sy \cdot y$

Shear (horizontal)



$x' = x + 0.7 \cdot y,\ y' = y$

Scaling acts relative to the origin, so scaling a shape that is not centered there will also shift its position. Scaling about an arbitrary point is therefore done by translating that point to the origin, scaling, and translating back.

$$M = T(p) \cdot S(sx, sy) \cdot T(-p)$$

scaling about the point p (standard form)

9 Combining Transformations

1

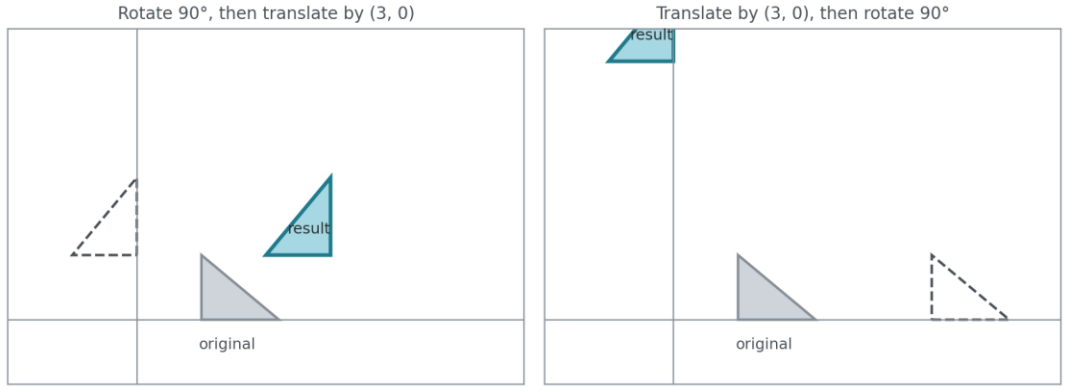
Combined by applying one after another. Because each transform can be written as a small matrix, a whole sequence — scale, then rotate, then translate — collapses into a single combined matrix that a computer can apply to every point in one step.

2

Order matters. Rotating and then translating an object is generally not the same as translating and then rotating it; graphics code always specifies transforms in the reverse of the order in which they conceptually apply to the object.

3

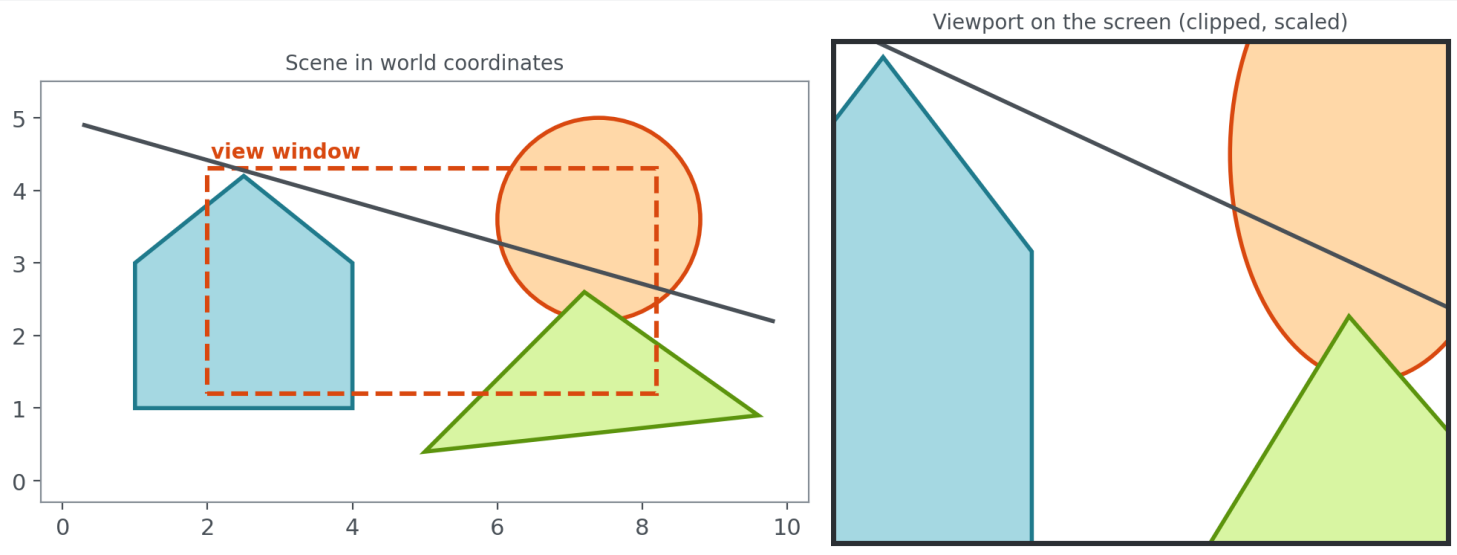
What is preserved. A transform built purely from translation and rotation preserves every length and angle in a shape; adding uniform scaling still preserves angles and proportions, just not absolute size.



$$p' = M \cdot p, \quad M = T \cdot R \cdot S$$

S is applied first, T last — written in reverse order

10 Windows and Viewports



View window: a rectangular region of the scene, expressed in whatever coordinates are natural for the scene.

Viewport: the region of the screen (or of an image) where that window will actually be drawn.

The window-to-viewport transform, itself just a combination of translation and scaling, maps every point inside the window to a corresponding point inside the viewport.

Transforming the window = transforming the objects. Moving, enlarging, or rotating the view window has exactly the same visual effect as moving, shrinking, or rotating every object in the scene in the opposite way. Mathematically, both are just transforms applied somewhere along the chain of coordinate changes that eventually produces pixel coordinates.

$$x_v = x_{vmin} + (x_w - x_{wmin}) \cdot (x_{vmax} - x_{vmin}) / (x_{wmax} - x_{wmin})$$

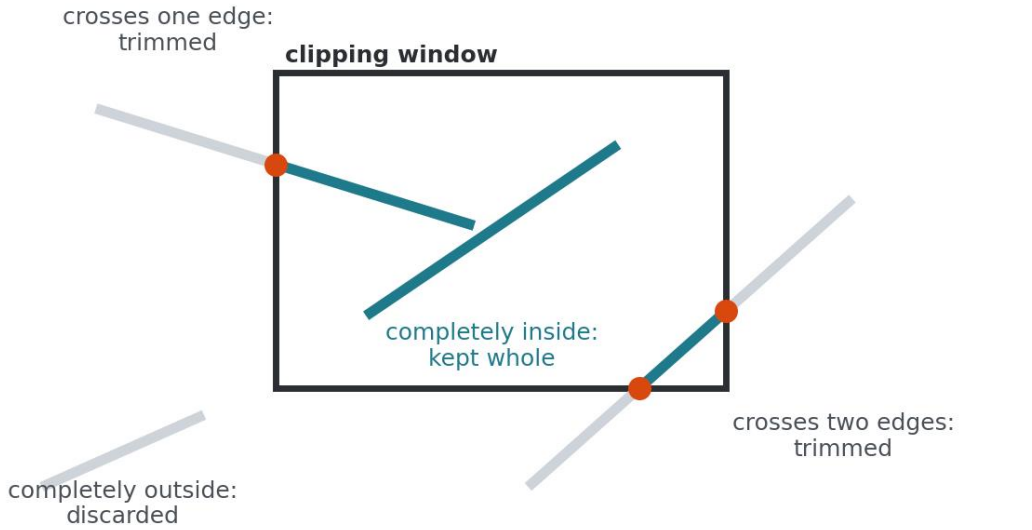
the same rule applies to y (standard form of the window-to-viewport mapping)

10 Clipping

Choosing a view window also raises the question of what to do with anything that falls outside it: parts of the scene beyond the window boundary should not be drawn at all.

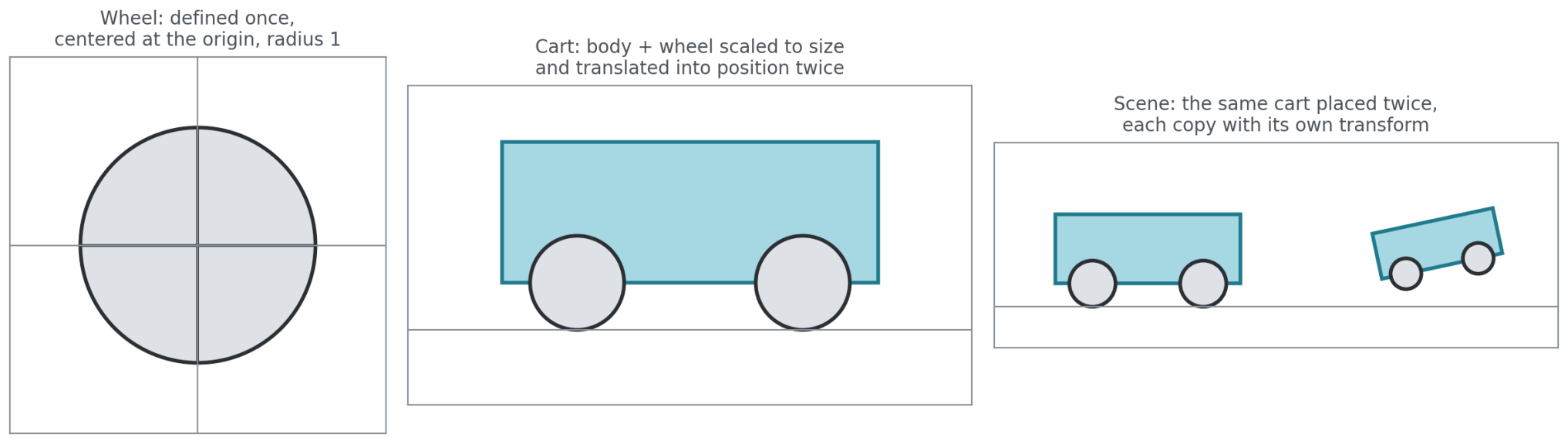
Deciding exactly which points, line segments, and polygon edges lie inside or outside a rectangular window — and, for the parts that straddle the boundary, computing precisely where the new, clipped edge should fall — is a surprisingly detailed problem in its own right.

It is the task handled by the point-, line-, and polygon-clipping algorithms covered later in this book.



11 Hierarchical Modeling

Complex pictures are rarely built primitive by primitive from scratch. Design each component in its own natural coordinate system — typically centered on the origin — and place a copy wherever it is needed.



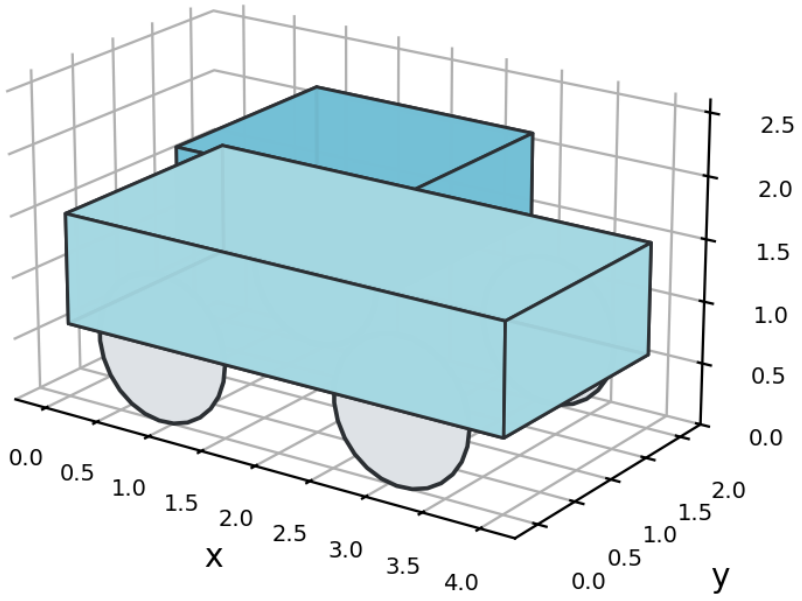
Modeling transform

Usually a scale, followed by a rotation, followed by a translation, places a copy of a component wherever it is needed.

Reuse and animation

The same cart can be placed into a larger scene multiple times, each with its own transform, and each copy can be animated independently by changing its transform slightly from one frame to the next.

12 From 2D to 3D: Modeling



The same foundational ideas extend directly into three dimensions: a scene is a collection of geometric primitives — often nothing more exotic than points, line segments, and triangles — positioned in a 3D world coordinate system given by x, y, and z.

As in 2D, complex objects are assembled hierarchically: a car can be modeled once as a body and one wheel, with the wheel placed at each of the car's four corners using four different modeling transforms, and several cars added to a scene the same way.

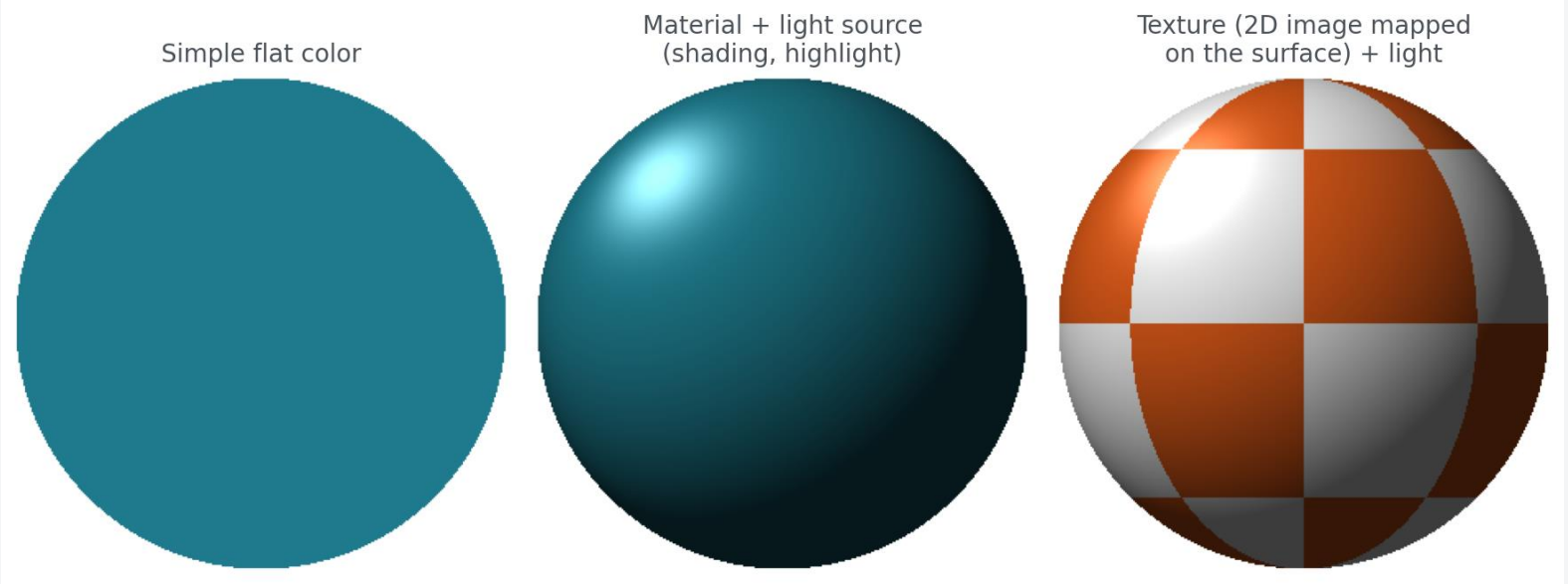
Translation, rotation, and scaling reappear essentially unchanged, just with an extra coordinate to carry along, and are represented uniformly using four-by-four matrices so that any sequence collapses into a single matrix per object.

$$\begin{bmatrix} 1 & 0 & 0 & tx \\ 0 & 1 & 0 & ty \\ 0 & 0 & 1 & tz \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4×4 translation matrix (standard form)

12 Materials, Textures, and Lights

Geometry alone is not enough to produce a convincing image; appearance also matters.



Material

In place of a simple flat color, 3D objects are usually given a material — properties describing how a surface's color, shininess, roughness, or transparency respond to light.

Texture

Typically a 2D image mapped onto the surface to add visual detail without modeling every bump and marking as separate geometry.

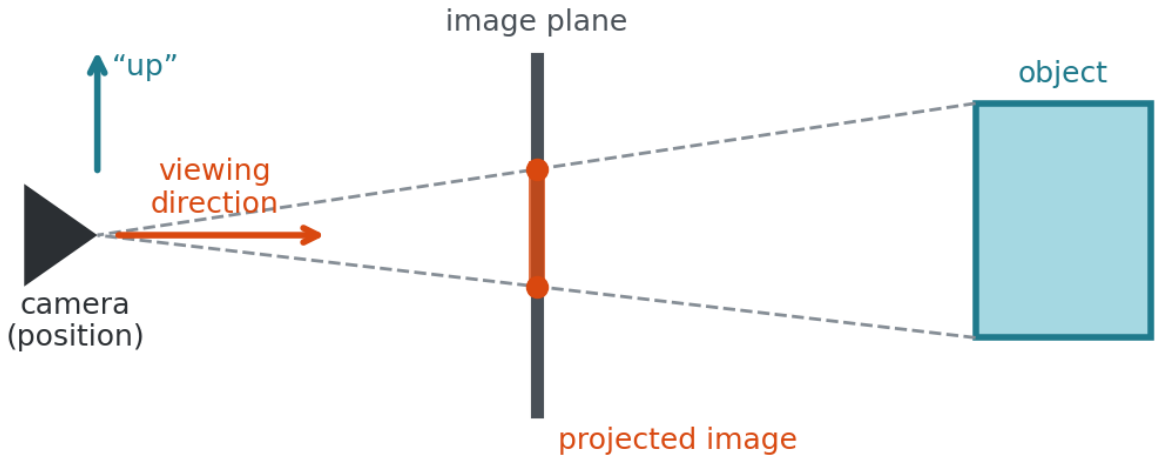
Light sources

Each has its own color, intensity, and position or direction; their interaction with surface materials determines the final shading of every visible point.

Source: IT311 Computer Graphics book, Chapter 1

12 Viewpoint and Projection

A three-dimensional scene has to be turned into the flat, two-dimensional image that actually appears on screen.



Choosing a viewpoint

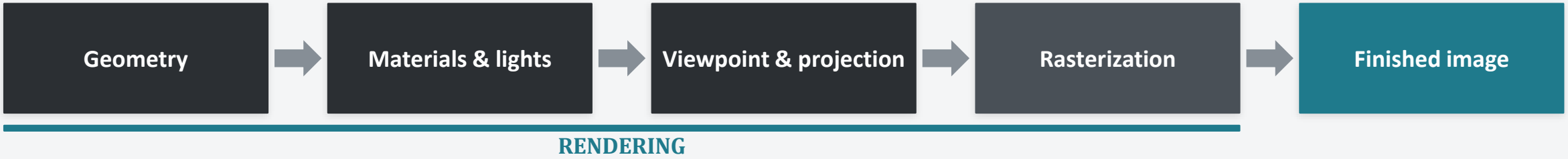
Essentially placing a virtual camera into the scene, with a position, a viewing direction, and an "up" direction.

Projecting

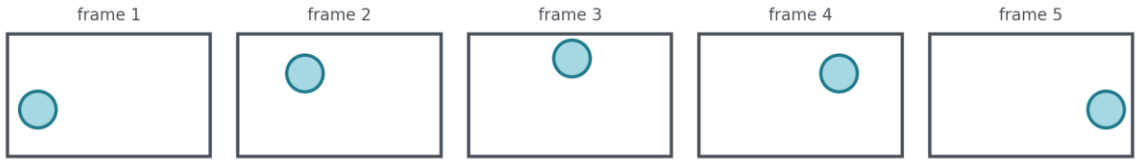
Projecting the scene from three dimensions down to two — the geometric equivalent of taking a photograph.

12 Rendering and Animation

The last step, assigning a final color to every pixel of that 2D image, is rasterization, and the entire process, from geometry and materials through to a finished image, is called rendering.



When the goal is an animation rather than a single picture, the whole pipeline is simply repeated for a sequence of frames, with small, systematic changes — to coordinates, transforms, materials, or the viewpoint — from one frame to the next.



This raster-projection pipeline, underpinned by the same translation, rotation, and scaling transforms introduced earlier, is what the final chapters of this book extend into full 3D.

Chapter Summary

Computer graphics turns a description of a scene into an array of colored pixels. The scene is described in world coordinates by basic shapes (strokes and fills) with attributes such as color; geometric transformations place and animate the shapes, a window selects the part of the world to show, clipping removes what lies outside it, and a viewport maps the result onto the screen, where rasterization decides which pixels each shape covers.

